Infosys Springboard Virtual Internship 7.0 Completion Report

Batch Number : Infosys Springboard 7.0 (confirm official batch)

Start date : 19 Aug 2026

Team Members :

- Md. Ayesha Afreen
- Shamya S Shetty
- Priaanka Avanigadda
- Durga Surya Teja
- Vankudothu Divya

Internship Duration: 8 Weeks (confirm official duration)

1. Project Title

MoodMentor – AI-Based Employee Wellness Management Platform

2. Project Objective

The main objective of MoodMentor is to develop a web-based employee wellness platform that helps users reflect on their emotional state through text-based emotion analysis and supportive wellness recommendations. Emotional well-being can affect daily routines, motivation, and workplace experience, while people may find it difficult to describe or track how they feel consistently.

MoodMentor accepts a user's text check-in, processes it using natural language processing and emotion-classification components, and presents an estimated emotion with related wellness guidance. The platform also includes views for reviewing emotion history, trends, and feedback. It is intended as a supportive self-reflection tool, not as a medical diagnostic system or a replacement for professional care.

3. Project description in detail

3.1 Overview

MoodMentor is a web application created during Infosys Springboard Virtual Internship 7.0. The React frontend provides the check-in and dashboard experience, a Python FastAPI backend exposes APIs for emotion analysis, history, trends, and feedback, and the data layer stores check-in and feedback records.

The project was developed in stages, covering the frontend interface, backend API integration, emotion-analysis experiments, recommendation logic, data handling, testing, and deployment.

3.2 System Functionality

A. Event Planning and Management

Users can enter a short text reflection about how they are feeling. The application sends the text for processing and displays the predicted emotion and related information returned by the backend.

B. Venue and Resource Coordination

The platform supports review of emotions detected from text. Emotion labels explored in the project include Joy, Sadness, Anger, Fear, Surprise, and Disgust. The prediction is a model estimate and may not perfectly represent a person's actual emotional state.

C. Conflict Detection and Smart Scheduling

MoodMentor uses NLP-based analysis to estimate an emotion from a text check-in. The model output is shown as supportive information and should not be treated as a clinical assessment.

D. Registration and Attendance

The check-in workflow accepts user-entered text and returns an emotion-analysis result. The interface is designed to make the result easy to review.

E. Vendor Management

The recommendation functionality connects the detected emotional state with supportive suggestions, such as a breathing or reflection activity. Recommendations are intended for general well-being support rather than diagnosis or treatment.

F. Budget and Expense Tracking

A trends view summarizes recorded emotion information so users can review patterns across their check-ins. The usefulness of these summaries depends on the amount and quality of recorded data.

G. External Service Integrations

The platform includes a feedback function for users to respond to recommendations. Feedback data can help the team review recommendation usefulness and identify opportunities for improvement.

H. Analytics, Insights and Reports

The history and feedback interfaces provide ways to review previous check-ins and feedback records, where those records are available in the application.

3.3 Technical Implementation

The system follows a modular web architecture consisting of a React/Vite frontend, a Python FastAPI backend, emotion-analysis and recommendation logic, and a data layer for check-in and feedback records.

The frontend provides the user interface for check-ins, analysis results, recommendations, history, trends, and feedback. The FastAPI backend handles requests and returns structured responses to the frontend.

The project uses Python for backend development and NLP experiments, React JS and Vite for the frontend, and API-based communication between the client and backend. The implementation includes emotion analysis, recommendation logic, data handling, and interface testing.

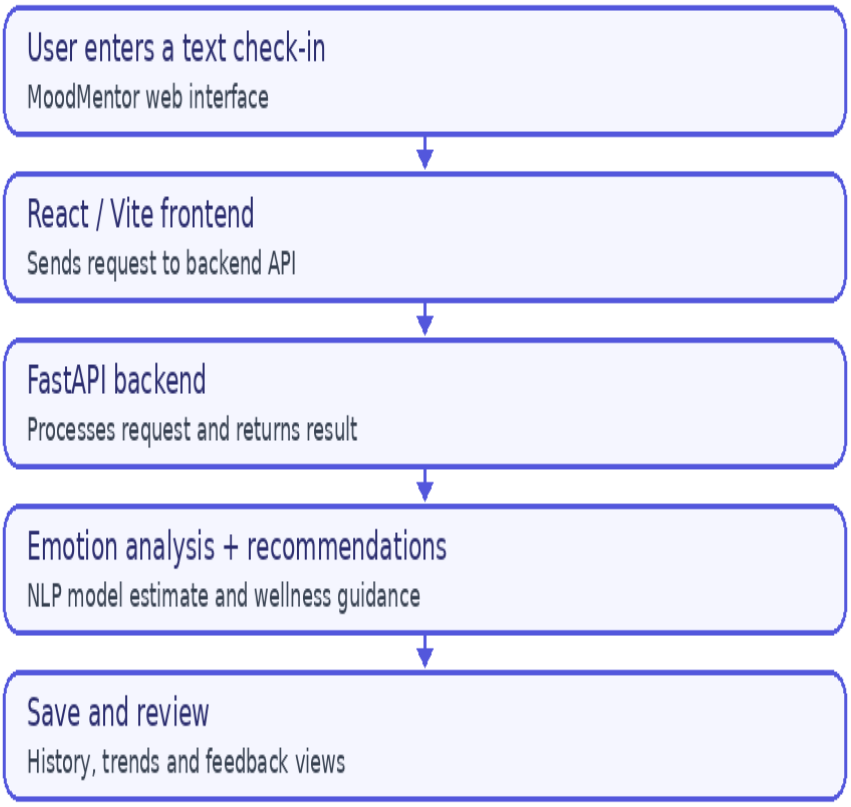
The modular structure helps separate user interaction, backend processing, and analysis logic. Further model evaluation, secure user authentication, and broader testing can improve the platform in future versions.

3.4 Technologies Used

Component	Technology
Programming language	Python, JavaScript
Backend framework	FastAPI
Frontend	React JS, Vite, HTML, CSS
Data / persistence	Check-in and feedback records (verify final database configuration)
API Communication	HTTP / REST-style API endpoints
NLP / emotion analysis	NLTK, VADER baseline; BERT and DistilBERT experiments
Emotion categories	Joy, Sadness, Anger, Fear, Surprise, Disgust
Recommendations	Hybrid approach explored using text similarity and emotion/category matching
Deployment Platform	Vercel frontend; verify backend hosting

Development tools	VS Code, Git, GitLab

3.5 Workflow of the System



4. Timeline Overview

Week	Activities Planned	Activities Completed
Week 1	Requirement analysis and project planning	Defined MoodMentor as a text-based employee wellness and emotion-analysis platform.
Week 2	Frontend design and environment setup	Created the React/Vite project structure and dashboard-related screens.

Week 3	Backend API development	Configured FastAPI endpoints for emotion analysis, history, trends, and feedback.
Week 4	Emotion-analysis experiments	Explored a VADER/NLTK baseline and transformer-based models for six emotion classes.
Week 5	Recommendation logic and data handling	Worked on recommendations and check-in/feedback data fields.
Week 6	Frontend-backend integration	Connected frontend views with backend endpoints and reviewed response handling.
Week 7	Testing and refinement	Checked application flows and reviewed model output limitations.
Week 8	Deployment and documentation	Deployed the frontend to Vercel, checked the site, and prepared the report.

5a. Key Milestones

Milestone	Description	Date Achieved
Project planning	Defined project scope and wellness check-in use case	Confirm date
Frontend prototype	Created initial React/Vite interface and dashboard	Confirm date
API integration	Connected frontend with FastAPI endpoints	Confirm date
Emotion analysis and recommendations	Integrated NLP experiments and supportive recommendation flow	Confirm date

Deployment and report	Deployed frontend and prepared final report	Confirm date

5b. Project execution details

MoodMentor was developed iteratively, with the interface, backend services, emotion-analysis components, and recommendation logic implemented and connected in stages. Development focused on presenting analysis results clearly and supporting check-in history, trends, and feedback.

During implementation, the team worked on connecting frontend screens to backend endpoints, defining consistent fields for text, emotion, confidence, intensity, sentiment, and feedback, and checking how results were displayed. NLP experiments also highlighted the need to evaluate model quality carefully.

The final stage involved deploying the frontend to Vercel, checking the website in a browser and on a mobile device, and preparing the technical report. The delivered version opens directly to the dashboard and does not include a login flow.

6. Snapshots / Screenshots

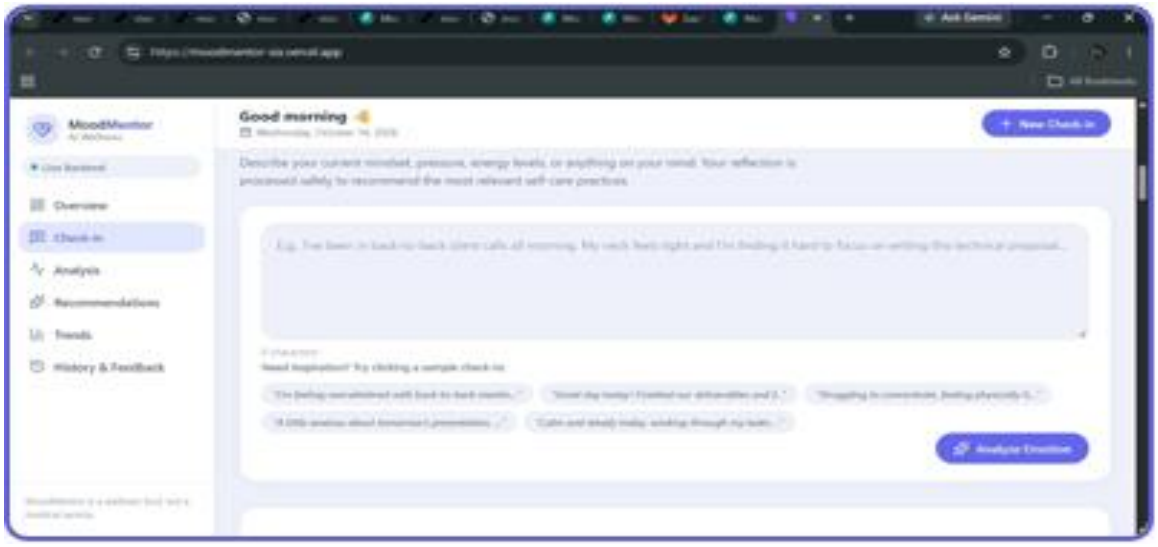
Main Dashboard

MoodMentor dashboard and navigation.



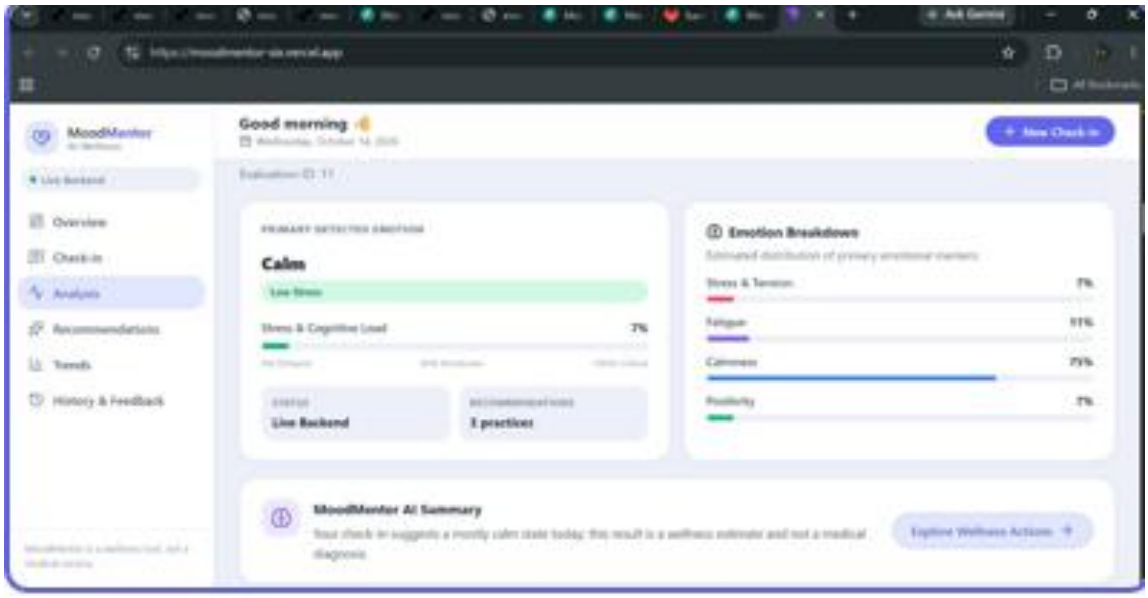
Emotion check-in interface for entering text.

Event list, event creation, or event details.



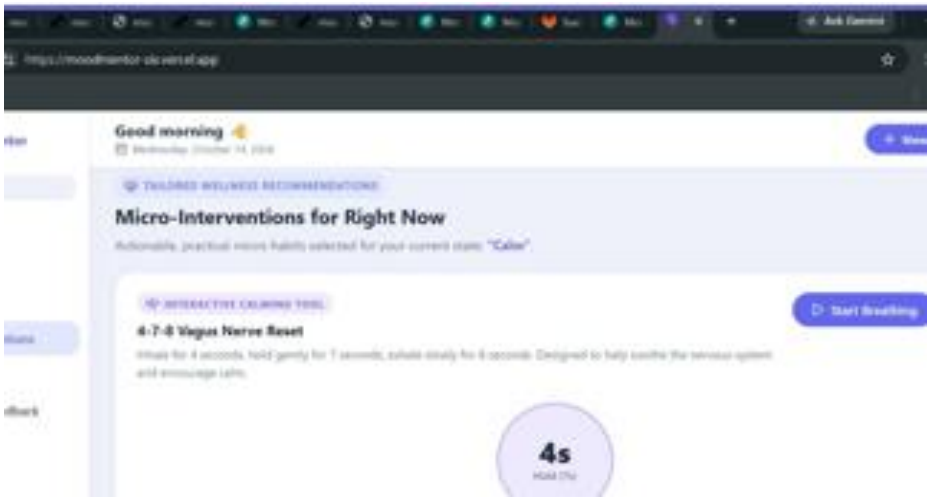
Emotion-analysis result and emotion breakdown.

Supportive wellness recommendation interface.



Trends dashboard with summary indicators and chart.

Public registration form or confirmation page



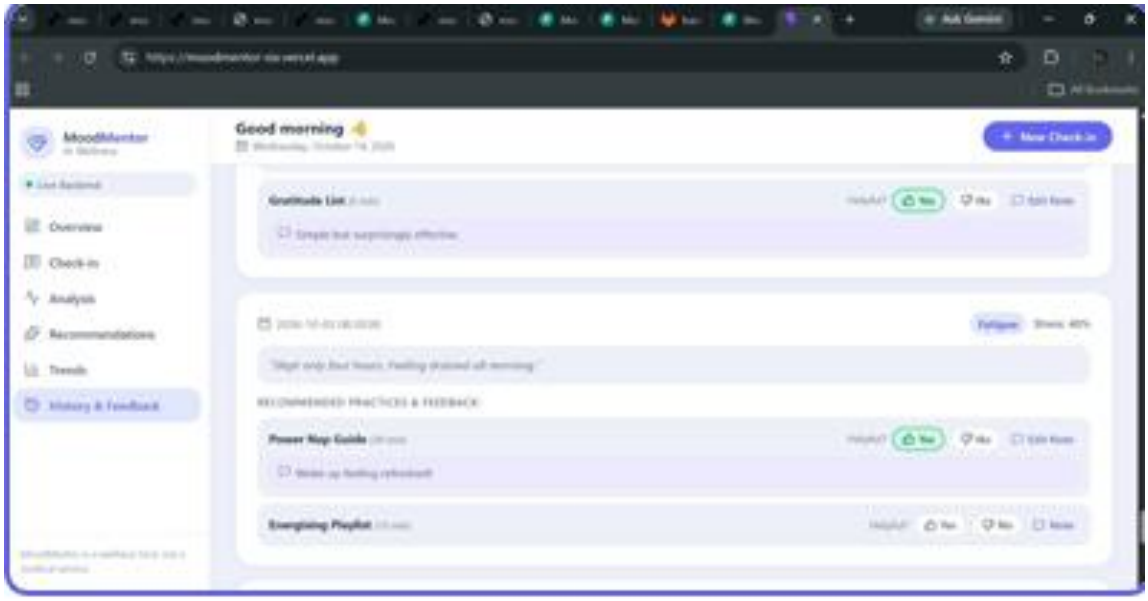
History and feedback interface.

Review of prior check-ins and recommendation feedback.



Analytics and Reports

Event metrics, analytics, or final reports page.



7. Challenges Faced

1. Backend and Frontend Integration

Connecting the React frontend with the FastAPI backend required careful handling of endpoint paths, request formats, response structures, and backend availability. API testing and configuration checks helped identify integration issues.

2. Database Management

The application needed consistent fields for check-in text, predicted emotion, confidence, intensity, sentiment, and feedback. Consistent API responses and data handling are important for showing reliable history and trends.

3. Event Registration and Payment Workflow

Emotion classification is challenging because short text can contain ambiguity, sarcasm, or mixed feelings. Model experiments required careful evaluation and showed that dataset preparation and further tuning remain important.

4. Attendance Tracking and Analytics

Displaying analysis results and trends required coordination between backend responses and frontend components. Testing helped check whether returned values were presented clearly.

5. Deployment and Configuration

Deployment required the frontend to use the correct backend API URL and the backend to be reachable for live functionality. Environment configuration and endpoint availability were important checks.

6. Testing and Debugging

Testing involved checking the website, API endpoints, and result presentation. Additional systematic testing and model evaluation remain useful improvements.

8. Learnings & Skills Acquired

1. Technical Skills

Gained practical experience with React JS and Vite for frontend development, Python and FastAPI for backend API development, and NLP concepts for sentiment and emotion analysis. The project also involved model experiments, recommendation logic, API integration, Git/GitLab, and Vercel deployment.

Developed practical knowledge of building an AI-enabled web application with check-ins, emotion analysis, recommendations, history, trends, and feedback. The project emphasized that model predictions are estimates and should be evaluated carefully.

2. Analytical and Problem-Solving Skills

Improved the ability to investigate API errors, check response data, debug frontend-backend communication, and recognize limitations in model evaluation. The work encouraged systematic testing rather than assuming every model prediction is correct.

3. Research and Development Skills

Practised comparing NLP approaches, reviewing classification outputs, connecting model results to an application workflow, and identifying opportunities for further evaluation and improvement.

4. Collaboration and Professional Skills

Developed experience in project coordination, technical communication, documentation, version control, and collaboration across frontend, backend, analysis, testing, and deployment tasks.

9. Testimonials from team

The MoodMentor project provided practical experience in developing an AI-enabled web application focused on employee wellness. It helped the team understand how a user interface, backend APIs, text-processing components, recommendation logic, and data views can be brought together into one application.

The development process strengthened our understanding of React frontend development, Python FastAPI services, NLP and emotion classification, API integration, testing, and deployment. It also highlighted the importance of evaluating AI outputs carefully and presenting them responsibly.

Working on the project improved problem-solving, debugging, testing, and software integration skills. It also demonstrated the importance of usability, clear documentation, responsible AI communication, and collaboration during application development.

10. Conclusion

MoodMentor – AI-Based Employee Wellness Management Platform demonstrates a web-based approach to wellness check-ins using text-based emotion analysis and supportive recommendations. The platform brings together a React/Vite frontend, FastAPI backend endpoints, NLP experiments, recommendation logic, and views for history, trends, and feedback.

The project provided practical experience in frontend development, backend API implementation, NLP experimentation, recommendation logic, data handling, testing, and deployment. It also reinforced that emotion predictions are estimates and must not be treated as medical diagnoses.

Overall, the project strengthened our understanding of developing an AI-enabled web application and identifying areas for improvement. Future enhancements may include better training data and model evaluation, secure user authentication and privacy controls, improved recommendations, stronger persistence and monitoring, and broader usability testing.

11. Acknowledgements

We express our sincere gratitude to Infosys Springboard for providing the opportunity to participate in the Virtual Internship 7.0 and work on an applied AI-enabled software project.

We thank our internship mentor and project guide for their guidance, feedback, and support throughout the project.

We also thank our team members for their collaboration, technical discussions, and contributions to development, testing, deployment, and documentation.